

# Pagination

Pagination is the process of splitting large result sets into smaller subsets to improve performance, reduce memory usage, and enhance user experience.

## ◆ Why Pagination Is Needed

1. **Performance** – Avoid loading millions of records at once
2. **Scalability** – Protects backend and database
3. **Network Efficiency** – Reduces payload size
4. **User Experience** – Enables smooth browsing (pages / infinite scroll)

Pagination controls data flow between client and server by limiting the number of records returned per request while maintaining consistent ordering.

Pagination is about:

Controlling data flow + protecting system performance + enabling scalable browsing

## What Makes Pagination “Correct”?

Pagination has three essential components:

### 1. Limit (How many records per request)

Example: 10 per request.

### 2. Position Indicator (Where to continue from)

This can be:

- Offset
- Page number
- Cursor
- ID

- Timestamp

### 3. Ordering (Extremely Important)

If ordering is not consistent, pagination breaks.

Example:

```
SELECT * FROM posts LIMIT 10;
```

Without ORDER BY — results are unpredictable.

Correct:

```
SELECT * FROM posts  
ORDER BY created_at DESC  
LIMIT 10;
```

Ordering ensures stability.

This is what makes your answer sound senior.

## Pagination Strategies

### 1 Offset-Based Pagination

#### What It Is

You tell the database:

Skip first N rows and give me the next M rows.

Example:

```
GET /users?offset=20&limit=10
```

SQL:

```
SELECT * FROM users  
ORDER BY created_at DESC  
LIMIT 10 OFFSET 20;
```

Meaning:

- Skip 20 rows
- Return next 10 rows

## How It Works Internally

Even if you skip 1 million rows, the database still:

- Reads them
- Counts them
- Discards them

It doesn't magically jump.

That's why it becomes slow for large offsets.

## When It Breaks

If new rows are inserted while user is paging:

User might:

- See duplicate data
- Miss some data

Because row positions shift.

## 2 Page-Based Pagination

This is just a wrapper over offset.

[GET /users?page=3&limit=10](#)

Backend converts:

$\text{offset} = (\text{page} - 1) * \text{limit}$

Technically identical to offset.

It only exists because UI prefers page numbers.

Performance characteristics = same as offset.

## 3 Cursor-Based Pagination

This is where engineering maturity begins.

Instead of saying:

Skip first 20 rows

We say:

Give me records after this specific record.

Example:

GET /posts?cursor=eyJpZCI6MTAwfQ== // here cursor is parade as encoded form in request.

Decoded cursor might contain:

{ "id": 100 } // this is index ID

SQL:

```
SELECT * FROM posts
WHERE id < 100
ORDER BY id DESC
LIMIT 10;
```

### **Why This Is Powerful**

Database uses index to directly jump to ID 100.

It does NOT scan from the beginning.

So performance is stable even at 100 million rows.

### **Why It's Consistent**

Even if new rows are inserted:

You're always saying:

Continue from this exact record.

So no shifting problem.

### **Limitation**

You can't easily jump to page 50.

Because you don't know where page 50 starts without walking through pages sequentially.

## Best For

- Social feeds
- Infinite scroll
- Large datasets
- High-performance APIs

### **Keyset Pagination (Advanced Form of Cursor)**

This is a more precise and optimized cursor strategy.

Instead of encoding random cursor logic, you use a stable indexed column (usually primary key).

Example:

`GET /orders?last_id=105`

SQL:

```
SELECT * FROM orders
WHERE id > 105
ORDER BY id ASC
LIMIT 20;
```

Extremely efficient if:

- id is indexed
- Ordering matches index

This is near  $O(1)$  access cost.

### **Time-Based Pagination**

Used when data is naturally time ordered.

Example:

`GET /logs?before=2026-03-02T10:00:00Z`

SQL:

```
SELECT * FROM logs
WHERE created_at < '2026-03-02T10:00:00Z'
```

ORDER BY created\_at DESC  
LIMIT 50;

Perfect for:

- Logs
- Chat messages
- Audit trails
- Transaction history

### **The Real Engineering Insight**

Pagination strategy is not chosen based on simplicity.

It is chosen based on:

1. Dataset size
2. Write frequency
3. Need for page numbers
4. Consistency requirements
5. Query indexing strategy

### **API Response Design (Important)**

Good pagination API should return:

```
{  
  "data": [...],  
  "pagination": {  
    "nextCursor": "abc123",  
    "hasNextPage": true,  
    "limit": 10  
  }  
}
```

## 1. Bidirectional Pagination

Support:

- next
- previous

Requires storing both start and end cursors.

## 2. Compound Cursor

Example:

Sort by:

ORDER BY created\_at DESC, id DESC

Cursor must encode both.

Because timestamps can be same.

## 3. Security

Never expose raw database internals blindly.

Use:

```
Buffer.from(JSON.stringify({id: 100})).toString('base64')
```

You're building:

A trading platform (you care about this domain).

You have:

10 million transactions.

Users:

- Scroll transaction history
- Filter by date
- Jump to specific month

👉 Which pagination strategy would you design and why?

Since transactions are time-series data, I would use time-based keyset pagination with a compound cursor (`created_at + id`). This ensures stable ordering, avoids deep offset scans, supports infinite scrolling efficiently, and remains performant even with tens of millions of rows. For month-based navigation, I would apply date range filtering and paginate within that range using the same cursor logic.